

Batch Gradient Descent

Contents

1. Overview	2
2. Gradient descent with many inputs	3
3. The derivatives	3
3.1 With two inputs	3
3.2 Any number of inputs	4
3.3 All derivatives at once	4
4. Batch gradient descent in code	5
5. Early stopping	5
6. Advantages and disadvantages	7
7. Summary	8
Sources	8
8. Key terms	8

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Gradient descent ([Note 57](#)).
- **Leads to:** Batch size in Keras ([Note 1020](#)).
- **Compare with:** Stochastic gradient descent ([Note 5](#)).

1. Overview

Key point: The three kinds of gradient descent differ only in how many rows they use for each update: all of them (batch), one (stochastic), or a small group (mini-batch).

Gradient descent comes in three main types. The update rule is the same in all three; they differ in how many training rows are used to compute each update (Figure 1).

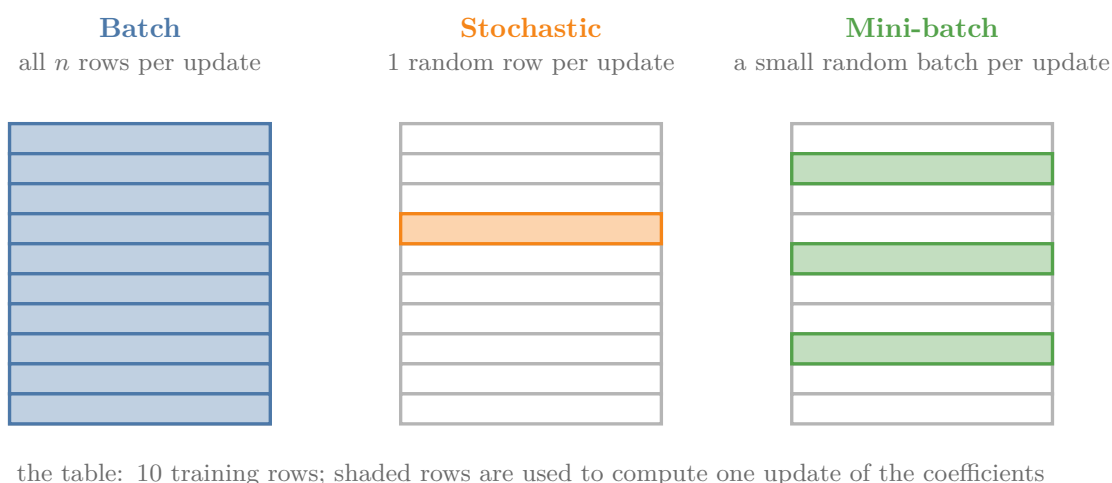


Figure 1: How many rows each type uses per update

Type	Rows per update	Updates per epoch (n rows)
Batch gradient descent	all n	1
Stochastic gradient descent (SGD)	1, chosen at random	n
Mini-batch gradient descent	a small random group, for example 32	n / batch size

The previous Note already used batch gradient descent, on one input. This Note extends it to any number of input columns, writes it with matrices, and codes it. The next two Notes cover the other two types.

2. Gradient descent with many inputs

Key point: With m inputs there are $m + 1$ coefficients, each with its own derivative; all are updated at the same step.

For multiple linear regression with m input columns, the model is

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_m x_{im}$$

and there are $m + 1$ coefficients to find. Batch gradient descent:

1. **Start** with any values, for example $\beta_0 = 0$ and every other $\beta_j = 1$.
2. **Each epoch:** compute the predictions for all rows, then the derivative of the loss with respect to every coefficient, then update all of them at once:

$$\beta_j \leftarrow \beta_j - \eta \frac{\partial L}{\partial \beta_j} \quad \text{for } j = 0, 1, \dots, m$$

3. **Repeat** for a fixed number of epochs.

The loss used here is the **mean** squared error, $L = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$. Dividing by n keeps the size of the derivatives independent of how many rows there are, so the same learning rate works for small and large datasets.

3. The derivatives

Key point: The derivative for coefficient j is the errors multiplied by column j of the data, averaged, times -2 . The intercept uses a column of 1s.

3.1 With two inputs

Key point: Writing out the loss for two inputs shows the pattern: each coefficient's derivative weights the errors by its own input.

With two inputs, $\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$. Differentiating the mean squared error with the chain rule, as in the previous Note:

$$\frac{\partial L}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

$$\frac{\partial L}{\partial \beta_1} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{i1}$$

$$\frac{\partial L}{\partial \beta_2} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{i2}$$

3.2 Any number of inputs

Key point: The general rule: for coefficient j , multiply each row's error by that row's value in column j .

In words: for coefficient β_j , take each row's error, multiply it by the row's value in column j , add these up, and multiply by $-2/n$.

$$\frac{\partial L}{\partial \beta_j} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) x_{ij}$$

The intercept fits the same rule if we imagine a column of 1s for it (as in the normal equation Note): multiplying by 1 changes nothing.

With numbers, for a tiny case of $n = 2$ rows with errors $(3, -1)$ and column j values $(2, 4)$:

$$\frac{\partial L}{\partial \beta_j} = -\frac{2}{2} (3 \times 2 + (-1) \times 4) = -(6 - 4) = -2$$

The derivative is negative, so this coefficient will increase.

3.3 All derivatives at once

Key point: All m derivatives together are one matrix product: $-\frac{2}{n} X^T (y - \hat{y})$.

Computing each derivative in a loop over rows and columns works but is slow in Python. The sum “errors times column j ” for every j at once is exactly a matrix product (Figure 2).

$$\underbrace{\frac{\partial L}{\partial \beta}}_{m \times 1} = -\frac{2}{n} \underbrace{X^T}_{m \times n} \underbrace{(y - \hat{y})}_{n \times 1}$$

one row per coefficient:
for coefficient j , the errors are
multiplied by column j of X , summed,
and divided by n

Figure 2: All derivatives as one matrix product

$$\frac{\partial L}{\partial \beta} = -\frac{2}{n} X^T (y - \hat{y})$$

X^T has one row per input column; multiplying it by the vector of n errors gives one number per column. Writing a computation as matrix operations instead of Python loops is called **vectorisation**. In the Notebook the vectorised derivative is many times faster than the loop even on this small dataset: between about 13 and 80 times in our runs, depending on the machine and the run.

4. Batch gradient descent in code

Key point: Each epoch: predict all rows, compute the errors, compute the intercept's and the coefficients' derivatives, update both.

Python: Batch gradient descent for multiple linear regression.

```
class GDRegressor:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.lr, self.epochs = learning_rate, epochs

    def fit(self, X, y):
        n = X.shape[0]
        self.intercept_ = 0.0
        self.coef_ = np.ones(X.shape[1])
        for _ in range(self.epochs):
            err = y - (X @ self.coef_ + self.intercept_)
            intercept_der = -2 * err.mean()
            coef_der = -2 * (X.T @ err) / n
            self.intercept_ -= self.lr * intercept_der
            self.coef_ -= self.lr * coef_der
        return self

    def predict(self, X):
        return X @ self.coef_ + self.intercept_
```

On the diabetes data (10 inputs, 353 training patients), with learning rate 0.5 and 1,000 epochs, training takes a few hundredths of a second, and the result is close to the exact OLS answer:

	Intercept	Test R^2
OLS (LinearRegression)	151.88	0.440
Batch gradient descent, 1,000 epochs	152.01	0.453

5. Early stopping

Key point: Stopping gradient descent early keeps the coefficients small. When a model has many input variables (features) for few records (observations), those small coefficients predict new data much better than the fully fitted OLS line.

The idea. We start every coefficient at zero. Each epoch moves the coefficients a little further from zero, towards the OLS answer. If we stop early, the coefficients stay small. Small coefficients cannot chase the noise in the training data, so stopping early acts like a brake on overfitting. For linear regression, stopping early does almost the same job as L2 (ridge) regularisation (Note 63), which pulls the coefficients towards zero (Goodfellow §7.8). Stopping on purpose when the score on held-out data is best is called **early stopping**.

When the brake matters. A brake only helps when the model would otherwise overfit. OLS overfits when there are many features for each observation and the target is noisy: the coefficients become badly determined and jump around from sample to sample (ESL §3.4.1). Here a **feature** is an input variable (one column of the data table), the **target** is the output we predict, and an **observation** is one record (one row).

The picture. Figure 3 changes one thing only: the number of features. Both panels use 200 diabetes patients for training and the rest for testing.

- **Left, 10 features:** the original 10 measurements. Test R^2 rises and then levels off at the OLS value. With 20 observations for every feature, OLS does not overfit, so there is nothing for the brake to fix.
- **Right, 65 features:** the same 10 measurements plus the square of each one and the product of every pair (the idea behind polynomial regression, [Note 61](#)). The target is noisy and there are now only about 3 observations per feature. Training R^2 keeps rising, but test R^2 peaks after about 40 epochs and then falls towards the poor OLS value.

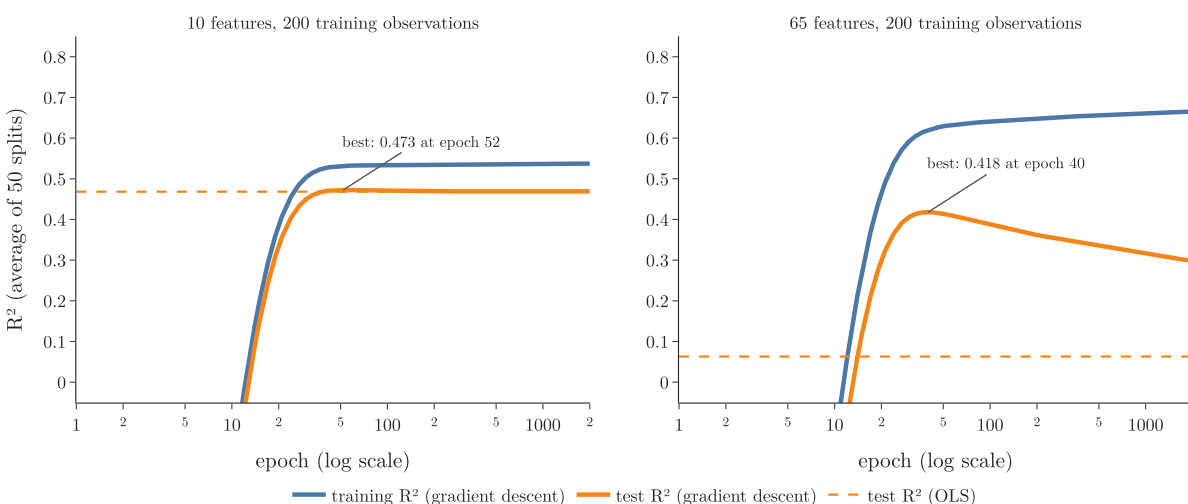


Figure 3: Training and test R^2 over the epochs, averaged over 50 random splits. Left: 10 features, where gradient descent ends at the OLS value. Right: 65 features, where test R^2 peaks early and then falls towards OLS.

The number. With 65 features, early stopping gives a test R^2 of 0.40 on average, against 0.06 for OLS. With the original 10 features, the two are equal (0.47).

Extra: How we measured this fairly (notebook, “Early stopping”). For each of 50 random train/test splits, we choose the stopping epoch on a validation part of the training data only, retrain on the whole training set for that many epochs, and then score once on the test set. The learning rate is 0.04 and the features are standardised.

Features	OLS test R^2	Early stopping test R^2	Early stopping better
10	0.468	0.467	26 of 50 splits
65	0.063	0.401	50 of 50 splits
65, with 350 training observations	0.332	0.432	48 of 50 splits

Early stopping keeps the coefficients short: with 65 features their length (the square root of the sum of their squares) is about 49 at the stopping epoch, against about 2,968 for OLS. More observations shrink the gain (last row), as expected: with more data OLS overfits less. Figure 3's peak of 0.418 is a little higher than 0.401 because the curve picks its best epoch with the test data itself; 0.401 is the honest figure.

Extra: Back to the 10-feature fit of section 4. Even after 50,000 epochs its coefficients are still far from OLS's (the largest gap is 319), although R^2 is almost identical. The largest gap is in the coefficient of s_1 , a blood measurement strongly related to another one, s_2 (correlation 0.895; multicollinearity, from the assumptions Note). With related features, the loss has a long, narrow valley along which many pairs of coefficients give almost the same loss (ISL §3.3.3, Figure 3.15). The notebook confirms the valley here: the flattest direction of the loss points mostly along s_1 , s_2 and s_3 (weights 0.71, -0.56 , -0.32), and the loss curves 447 times more steeply in its steepest direction than in this flattest one. A step size small enough for the steep direction makes slow progress in the flat one (Goodfellow §4.3.1), the same effect as with unscaled inputs in the previous Note.

6. Advantages and disadvantages

Key point: Batch gradient descent is smooth and stable, but each update needs every row, which is slow and memory-hungry on large data.

Advantages:

- **Stable:** every update uses the exact gradient of the whole training loss, so the loss goes down smoothly.
- **Vectorised:** the whole update is a few matrix products, which libraries compute very fast.

Disadvantages:

- **Slow on large data:** one update needs a pass over all n rows. With millions of rows, each step is expensive, and many steps are needed.
- **Memory:** the whole dataset must fit in memory at once for the matrix product.

These two problems are what stochastic and mini-batch gradient descent solve, in the next two Notes.

7. Summary

Item	Batch gradient descent
Rows per update	all n
Derivative of intercept	$-\frac{2}{n} \sum (y_i - \hat{y}_i)$
Derivative of coefficient j	$-\frac{2}{n} \sum (y_i - \hat{y}_i) x_{ij}$
All at once	$-\frac{2}{n} X^T (y - \hat{y})$
Early stopping, 65 features	test R^2 0.40 (OLS 0.06), average of 50 splits

- The three types of gradient descent differ only in how many rows feed each update.
- Each coefficient's derivative weights the errors by its own input column.
- Vectorised code computes every derivative with one matrix product.
- Stopping early keeps the coefficients small; with many features per observation, it predicts new data far better than OLS.
- Batch gradient descent is stable but needs the whole dataset for every step.

Sources

- **Goodfellow:** I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016 (deeplearningbook.org).
- **ESL:** Hastie, Tibshirani, Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009.
- **ISL:** James, Witten, Hastie, Tibshirani, *An Introduction to Statistical Learning*, 2nd ed., Springer, 2021.

8. Key terms

Term	Meaning
Batch gradient descent	Gradient descent that uses all training rows for every update
Stochastic gradient descent (SGD)	Gradient descent that uses one random row for every update
Mini-batch gradient descent	Gradient descent that uses a small random group of rows for every update
Mean squared error loss	The average squared error; its derivatives do not grow with the number of rows
Vectorisation	Writing a computation as operations on whole arrays instead of Python loops
Early stopping	Stopping training when the score on held-out data is best, before full convergence